15-288 Machine Learning Cheatsheet

Maximally condensed summary of the notes

Core Setup

Learning problem: From samples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, learn $f : X \to Y$ that minimizes error on unseen data.

Tasks:

- **Classification:** Y discrete; predict a class.
- **Regression:** $Y \subseteq \mathbb{R}^m$ continuous; predict a value.
- **Unsupervised:** no labels; cluster, compress, discover structure.
- **RL:** learn a policy maximizing cumulative reward.

Pipeline: Define task/metric → collect and clean data → EDA → engineer features → choose hypothesis class → train → validate/tune → test once → deploy/monitor.

Notation and Generalization

- **Hypothesis class** $\mathcal{H}$: candidate functions; its shape is the model bias.
- **Loss** $\ell(\hat{y}, y)$: error on one example.
- **Empirical risk** $L_{\text{emp}}(f) = \frac{1}{N}\sum_{i=1}^N \ell(f(\mathbf{x}^{(i)}), y^{(i)})$.
- **Generalization** means low expected loss on unseen samples from the same distribution.
- **Inductive bias** is necessary: many hypotheses fit training data; we prefer simpler/useful ones.

Train/val/test: Fit parameters on train, tune choices on validation/CV, report final performance once on test. Never let test influence training.

Paradigm Comparison

	Supervised	Unsupervised	Reinforcement
Data	labeled $(\mathbf{x}, y)$	unlabeled $\mathbf{x}$ only	generated by interaction
Signal	label error	similarity/structure	reward/cost
Goal	predict y	discover structure	learn policy
Examples	class/regression	clustering, PCA, density	control, games, robotics

Label scarcity: Labels are expensive, slow, noisy, and often require experts; that motivates unsupervised, self-supervised, and transfer-learning pipelines.

Supervised Learning Essentials

Losses and Metrics

Classification losses:

- 0/1 loss: $\ell(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$.
- Cross-entropy (binary): $-\big(y \log p + (1-y)\log(1-p)\big)$.
- Cross-entropy (multi-class): $-\sum_k y_k \log p_k$.
- Hinge loss: $\max(0, 1 - y(\mathbf{w}^\top \mathbf{x} + b))$ for $y \in \{-1, +1\}$.

Regression losses:

- SSE $= \sum_i (y_i - \hat{y}_i)^2$.
- MSE $= \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$.
- RMSE $= \sqrt{\text{MSE}}$.
- MAE $= \frac{1}{N}\sum_i |y_i - \hat{y}_i|$; more robust to outliers.

Confusion-Matrix Metrics

For binary classification:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}.$$

Use **accuracy** for balanced classes, **precision** when false positives hurt, **recall** when false negatives hurt, F_1 for imbalanced positive-class detection, **ROC-AUC/PR-AUC** when threshold-free ranking matters.

Overfitting and Model Selection

- **Overfitting:** training error low but validation/test error high.
- **Underfitting:** both errors high; model too simple or badly trained.
- **Bias–variance tradeoff:** more flexible models reduce bias but raise variance.
- Choose complexity at the validation minimum, not at best training score.

Generalization gap: $L_{\text{gen}} - L_{\text{emp}}$ is the quantity we care about, but true L_{gen} is unknown; validation/test error estimates it. **Promote generalization:**

- Match model capacity to the data size/noise level.
- Add regularization terms $\lambda R(\mathbf{w})$ or early stopping.
- Gather more representative data when possible.
- Use correct train/val/test separation and robust validation protocols.

Margin-Based View of Classification

For binary labels $y \in \{-1, +1\}$ and score $f(\mathbf{x})$, the margin is

$$\mu(\mathbf{x}) = y\, f(\mathbf{x}).$$

- $\mu > 0$: correct sign; $\mu < 0$: misclassified.
- Larger positive margin means more confident correct prediction.
- 0/1, hinge, logistic, and exponential losses are all surrogate losses on μ.

Validation and Workflow

Hold-Out and Cross-Validation

- **Hold-out:** split once (e.g. train/val/test); fast but noisy.
- **K-fold CV:** split into K folds, train on $K-1$, validate on the last, average scores.
- Typical choice: $K = 5$ or 10.
- Use CV for model/hyperparameter selection, then retrain on full train set.

Leakage rule: Any operation that learns from data (imputation, scaling, PCA, feature selection, target encoding) must be fit on training folds only, then applied to validation/test.

Design Checklist

1. Define output space, metric, and operating constraints first.
2. Pick a hypothesis class with the right inductive bias.
3. Choose preprocessing compatible with the model.
4. Select a loss aligned with the business/scientific objective.
5. Tune hyperparameters only with validation/CV.
6. Interpret results through errors, calibration, and failure cases.

Hypothesis Classes

- **Parametric:** finite parameters, fixed form (linear/logistic regression, neural nets with fixed architecture).
- **Nonparametric:** complexity grows with data (k-NN, trees).
- **Consistency** on train does *not* imply generalization.

Data Preprocessing

Outliers

Why care: Outliers distort means, variances, correlations, distances, regression slopes, and gradient updates.

- **Z-score:** $z = \frac{x-\mu}{\sigma}$; flag if $|z| > 3$ (assumes near-Gaussian data).
- **Modified Z-score:** $0.6745\frac{x-\text{median}}{\text{MAD}}$; robust.
- **IQR rule:** $\text{IQR} = Q_3 - Q_1$; outlier if $x < Q_1 - 1.5\,\text{IQR}$ or $x > Q_3 + 1.5\,\text{IQR}$.
- **Percentile clipping/winsorization:** cap extremes at chosen quantiles.

Handle by: fixing data-entry errors, deleting impossible points, clipping, transforming, robust scaling, or using robust models. Remove only with reason.

Scaling

$$\text{Standardize: } x' = \frac{x-\mu}{\sigma} \qquad \text{Min–max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

$$\text{Robust scale: } x' = \frac{x - \text{median}}{Q_3 - Q_1} \qquad \text{MaxAbs: } x' = \frac{x}{\max|x|}$$

Needed for: distance-based methods (k-NN, clustering), margin/dot-product methods (SVM, logistic regression), gradient-based models, PCA, neural nets. Usually not needed for trees/random forests.

Feature Types and Encoding

- **Numerical:** continuous/discrete quantities.
- **Categorical nominal:** no order; use one-hot.
- **Categorical ordinal:** order exists; integer/ordinal encoding can be valid.
- **Binary:** 0/1 directly.

Label encoding: Maps categories to integers; dangerous for nominal categories because it injects fake ordering.

One-hot encoding: Adds one binary feature per category. Avoid exact linear dependence by dropping one level in strict linear models (dummy-variable trap).

Target encoding: Replace category by target mean; powerful but leakage-prone, so compute only within training folds.

Correlation and Feature Engineering

Pearson correlation:

$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \in [-1, 1].$$

- $r \approx 1$: strong positive linear relation; $r \approx -1$: strong negative; $r \approx 0$: no linear relation.
- Correlation *is not* causation and misses nonlinear dependence.
- Highly correlated features can create multicollinearity in linear models.
- Polynomial features allow linear models to fit nonlinear trends by expanding inputs (e.g. x, x^2, x^3, xy).

Practical rule: Use correlation matrices to remove redundant numeric features, but do not drop features solely by low correlation if nonlinear models or interactions may matter.

Image Data and Classical Vision Features

Image Representation

An image is a tensor: grayscale $H \times W$, color $H \times W \times C$ with channels such as RGB. Pixel intensities are features; resolution, channel order, and dtype matter.

- **Metadata (EXIF):** timestamps, camera settings, orientation, GPS; can be informative or leakage-prone.
- **Color spaces:** RGB for display, grayscale for intensity, HSV for hue/saturation separation.
- **Global statistics:** channel means, variances, min/max, entropy.

Histograms and Patches

- **Color histograms:** count pixel intensities per bin; robust to small translations but lose exact spatial layout.
- **Spatial patching:** compute local features on tiles to preserve rough spatial structure.
- **Combined descriptors:** concatenate global color/texture/edge summaries into one vector.

Filtering and Convolution

For image I and kernel K, cross-correlation at location (i, j) is the weighted sum of a local patch. Convolution flips the kernel before sliding; in deep learning, people often say "convolution" while computing cross-correlation.

- **Padding** preserves border size; **stride** controls step size.
- Common kernels: blur/smoothing, sharpen, edge detectors, emboss.
- Linear filters combine by addition; median filters are nonlinear and robust to salt-and-pepper noise.

Edges, Gradients, and HOG

Image gradients measure intensity change:

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}\right), \qquad \|\nabla I\| = \sqrt{I_x^2 + I_y^2}, \qquad \theta = \tan^{-1}(I_y/I_x).$$

- **Sobel operators** approximate horizontal/vertical derivatives.
- **Canny pipeline:** smooth $\to$ gradient $\to$ non-max suppression $\to$ hysteresis thresholding.
- **HOG:** bin local edge orientations, weight by magnitude, normalize over blocks.

- HOG is useful for classical object detection and shape-sensitive tasks.

Thresholding, Corners, and Matching

- **Global thresholding:** one cutoff for all pixels.
- **Adaptive thresholding:** threshold depends on local neighborhood.
- **Otsu:** choose threshold maximizing between-class variance.
- **Corners:** points with strong intensity variation in two directions (e.g. Harris detector).
- **SIFT/ORB:** keypoint descriptors for matching across scale/rotation/view changes.
- **Template matching:** slide a patch and compute similarity; simple but sensitive to scale/pose/illumination.

Classical image pipeline: preprocess → extract descriptors (histogram/HOG/keypoints) → normalize → feed into SVM/logistic/tree model.

Linear Regression and OLS

Model and Matrix Form

Linear model:

$$\hat{y} = w_0 + \sum_{j=1}^{d} w_j x_j = \mathbf{w}^\top \tilde{\mathbf{x}},$$

where $\tilde{\mathbf{x}}$ includes a leading 1 for the bias term. With design matrix $X \in \mathbb{R}^{N\times(d+1)}$,

$$\hat{\mathbf{y}} = X\mathbf{w}, \qquad L(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2.$$

Interpretation: Linear in parameters, not necessarily in raw inputs (polynomial features keep model linear in $\mathbf{w}$).

Normal Equations

For least squares,

$$\nabla_{\mathbf{w}} L = 2X^\top(X\mathbf{w} - \mathbf{y}) = 0 \quad \Rightarrow \quad X^\top X\mathbf{w} = X^\top \mathbf{y}.$$

If $X^\top X$ is invertible,

$$\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}.$$

- MSE is convex in $\mathbf{w}$ for linear regression.
- Closed form is elegant but costly/unstable for very large or collinear features.
- Gradient methods scale better in modern settings.

Regularization

- **Ridge:** minimize $\|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_2^2$; shrinks weights smoothly.
- **Lasso:** minimize $\|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_1$; can force exact zeros.
- **Elastic Net:** combine L_1 and L_2.

Regularization reduces variance, combats multicollinearity, and improves generalization.

Linear Models as Feature Machines

- Linear decision functions can still model nonlinear boundaries after a feature map $\phi(\mathbf{x})$.
- Basis expansions: polynomials, radial basis functions, handcrafted image descriptors, embeddings.
- Most of classical ML can be seen as: engineer a representation first, then fit a simpler model on top.

k-Nearest Neighbors

Idea: Store training set; for a query point, find the k closest examples.

- **Classification:** majority vote among neighbors.
- **Regression:** average/weighted average neighbor targets.
- Common distances: Euclidean $\|\mathbf{x} - \mathbf{x}'\|_2$, Manhattan $\|\mathbf{x} - \mathbf{x}'\|_1$.

Hyperparameter: Small $k \Rightarrow$ low bias/high variance; large $k \Rightarrow$ smoother boundary, higher bias/lower variance.

- Requires scaling.
- Prediction is expensive; training is trivial.
- Suffers in high dimension (distance concentration / curse of dimensionality).

Weighted neighbors: Distance-weighted voting/regression gives nearer points more influence and often smooths ties.

Optimization and Gradient Methods

Gradient Descent

Update rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L(\mathbf{w}_t),$$

where η is the learning rate.

- Too small η: slow convergence.
- Too large η: oscillation/divergence.
- Stop by validation plateau, gradient norm, or max epochs.

Variants:

- **Batch GD:** one gradient over full dataset; stable but slow.
- **SGD:** one example at a time; noisy but cheap and can escape shallow regions.
- **Mini-batch GD:** practical default; vectorized and reasonably stable.

Advanced Optimizers

- **Momentum:** accumulates velocity to smooth descent.
- **RMSProp:** adapts step size using running squared gradients.
- **Adam/AdamW:** momentum + adaptive scaling; strong default for deep learning.

Deep-learning defaults: mini-batches, shuffled data, Adam/AdamW, validation monitoring, early stopping, learning-rate schedules.

Optimization Pitfalls

- Nonconvex objectives can contain saddles and many local minima.
- Ill-conditioned curvature causes zig-zagging and slow training.
- Feature scaling and normalization often improve optimization as much as model choice.

Linear Classification

Linear Decision Functions

General linear classifier:

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \operatorname{sign}(g(\mathbf{x})) \text{ or } \arg\max_k g_k(\mathbf{x}).$$

Decision boundary is the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$. Linear models work well when classes are approximately linearly separable or after good feature engineering.

Logistic Regression

Binary logistic regression models

$$p(y = 1 \mid \mathbf{x}) = \sigma(z), \qquad z = \mathbf{w}^\top \mathbf{x} + b, \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

$$\log \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b$$

is the log-odds (logit). Predict class 1 when $p \geq \tau$ (often $\tau = 0.5$).

Loss: Minimize binary cross-entropy, often plus regularization.

Why likelihood: Logistic regression is equivalent to maximum conditional likelihood for Bernoulli labels.

Interpretation: Increasing feature x_j by 1 changes log-odds by w_j (holding others fixed).

Thresholding: Decision threshold need not be 0.5; raise threshold for higher precision, lower it for higher recall.

Multi-class: Softmax regression:

$$p(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad z_k = \mathbf{w}_k^\top \mathbf{x} + b_k.$$

$$\ell_{\text{CE}} = -\sum_k y_k \log p_k.$$

SVM

Geometric margin: For hyperplane $(\mathbf{w}, b)$ and example $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, +1\}$:

$$\gamma_i = \frac{y_i(\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|}.$$

Support vectors are points lying on/inside the margin and determine the boundary.

Hard-margin SVM:

$$\min_{\mathbf{w}, b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \ \forall i.$$

Soft-margin SVM:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \ \xi_i \geq 0.$$

Equivalent viewpoint: regularized hinge-loss minimization.

Kernel trick: Replace inner products by $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$ to get nonlinear boundaries without explicitly constructing ϕ. Common kernels: linear, polynomial, RBF.

When kernels help: Small/medium datasets with nonlinear boundaries and meaningful similarity structures; usually less attractive at very large scale.

Logistic vs. SVM: Logistic gives calibrated probabilities; SVM optimizes margin and often sharper boundaries.

Decision Trees

Idea: Recursively partition feature space into axis-aligned regions, predict by leaf majority class or leaf mean.

Split Criteria

- **Entropy:** $H(S) = -\sum_c p_c \log_2 p_c$.
- **Information gain:** $IG(S, \text{split}) = H(S) - \sum_j \frac{|S_j|}{|S|} H(S_j)$.
- **Gini impurity:** $G(S) = 1 - \sum_c p_c^2$.
- **Regression trees:** choose split with largest variance/MSE reduction.

Continuous features: Test thresholds of the form $x_j \leq t$ vs. $x_j > t$.

Pros, Cons, and Pruning

- Pros: interpretable, handles mixed features, no scaling required, nonlinear boundaries.
- Cons: high variance, greedy, unstable to data perturbations, can overfit deeply.
- **Pre-pruning:** max depth, min samples split/leaf, max leaves.
- **Post-pruning:** cost-complexity pruning trims weak subtrees.

Ensembles: Bagging/random forests reduce variance via averaging many trees; boosting fits sequentially to residual errors. **Feature importance caveat:** Tree importances can be biased toward high-cardinality features; permutation importance is often safer.

Unsupervised Learning

Clustering

Goal: Partition unlabeled data into groups so points in the same cluster are more similar than points in different clusters.

- Choice of similarity metric is an inductive bias.
- Number of clusters is often unknown and must be selected/estimated.
- Cluster labels are arbitrary; cluster quality depends on the problem definition.

k-means: Alternate between assigning each point to nearest centroid and recomputing centroids as cluster means; minimizes within-cluster SSE.

- Fast and common, but assumes roughly spherical clusters under Euclidean distance.
- Sensitive to scaling, initialization, and outliers.

Dimensionality Reduction

- High-dimensional data often lies near a lower-dimensional manifold.
- **PCA:** find orthogonal directions of maximum variance; useful for compression, denoising, visualization, decorrelation.
- Must scale features appropriately before PCA when units differ.

Generative Modeling View

- Generative models learn/discuss the data distribution $p(\mathbf{x})$ or joint structure in data.

- Useful for synthesis, density estimation, compression, anomaly detection, and representation learning.

Neural Networks

Perceptrons and MLPs

One neuron computes

$$a = \mathbf{w}^\top \mathbf{x} + b, \qquad h = \phi(a),$$

where ϕ is an activation function. Stacking layers gives an MLP:

$$\mathbf{x} \to h^{(1)} \to h^{(2)} \to \cdots \to \hat{y}.$$

- Without nonlinear activations, stacked linear layers collapse to one linear map.
- Hidden layers learn intermediate features.
- Universal approximation: sufficiently wide networks can approximate many functions, but optimization/generalization still matter.

Activations and Output Layers

- **Sigmoid:** $(1+e^{-z})^{-1}$; bounded, but saturates and causes vanishing gradients.
- **Tanh:** zero-centered but still saturates.
- **ReLU:** $\max(0, z)$; simple and dominant default for hidden layers.
- **Leaky ReLU/GELU:** help with dead neurons/smoother behavior.
- **Output:** identity for regression, sigmoid for binary class, softmax for multi-class.

Backpropagation

Backprop computes gradients layer-by-layer using the chain rule. For layer ℓ:

$$\delta^{(\ell)} = \frac{\partial L}{\partial a^{(\ell)}} = \left(W^{(\ell+1)\top} \delta^{(\ell+1)}\right) \odot \phi'(a^{(\ell)}).$$

Then

$$\frac{\partial L}{\partial W^{(\ell)}} = \delta^{(\ell)} h^{(\ell-1)\top}, \qquad \frac{\partial L}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

Training Dynamics

- **Epoch:** one full pass through training data.
- **Batch size:** examples processed before one update.
- Problems: vanishing/exploding gradients, overfitting, poor conditioning.
- Fixes: normalization, good initialization, ReLU-like activations, residual ideas, regularization.

Regularization in NNs: weight decay, dropout, data augmentation, early stopping, batch/layer normalization.

Convolutional Neural Networks

Why CNNs

MLPs ignore image locality and use too many parameters on pixels. CNNs exploit:

- **Local connectivity:** filters inspect small receptive fields.
- **Weight sharing:** one filter scans all spatial positions.
- **Translation-sensitive feature maps:** same detector reused across image.

Main Layers

- **Convolution:** sliding kernels produce feature maps.
- **Activation:** usually ReLU/GELU.
- **Pooling:** downsample (max/avg) to reduce spatial size and add invariance.
- **Flatten / global pooling + dense head:** final classification/regression stage.

Parameter efficiency: A small set of filters can replace enormous fully connected layers by reusing weights over space.

Transfer Learning

Use a pretrained CNN as:

- **Fixed feature extractor:** freeze backbone, train new head.
- **Fine-tuned model:** unfreeze some/all deeper layers and adapt to new data.

Especially useful when target dataset is small.

Modern NLP and Generative AI

Basic NLP Representations

- **Tokenization:** split text into tokens (character, word, or subword).
- **Subword tokenization** is standard: balances vocabulary size and OOV handling.
- **BoW/one-hot:** sparse, ignore order.
- **Embeddings:** dense vectors that encode semantic similarity.

Transformer Core

Transformers replace recurrence with attention and parallel sequence processing. **Scaled dot-product attention:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- **Queries, keys, values** are learned projections of token embeddings.
- **Positional encoding** injects order information.
- **Residual + normalization** stabilize training.
- **Feed-forward blocks** add nonlinear transformation per token.

Transformer Families

- **Encoder-only (BERT):** bidirectional context; best for understanding/classification.
- **Decoder-only (GPT):** causal masking; predicts next token autoregressively.
- **Encoder–decoder (T5):** seq2seq tasks like translation/summarization.

Typical workflow: tokenize → map to embeddings → run model → fine-tune/use pretrained head → evaluate with task metric.

Generative Models

- **Autoencoder:** encoder compresses to latent code, decoder reconstructs; optimize reconstruction loss.
- **GAN:** generator tries to fool discriminator; discriminator distinguishes real vs fake.

- **LLM generation controls:** temperature controls randomness; top-k/top-p restrict candidate next tokens.

High-Yield Comparisons

Algorithm Selection Heuristics

Model	Strengths	Weaknesses
Linear/Logistic	fast, interpretable, strong baseline	misses complex nonlinear patterns
k-NN	simple, nonparametric, flexible boundary	slow prediction, scaling sensitive, weak in high d
SVM	strong margins, effective in medium-sized/high-d data, kernels	expensive on large N, less interpretable
Trees	interpretable, mixed data, no scaling	unstable, overfit alone
Ensembles	strong accuracy, robust	less interpretable, heavier compute
MLP	flexible universal approximator	tuning/data hungry, weak inductive bias on images/text
CNN	excellent on images/spatial data	architecture/training complexity
Transformers	dominant on sequence/text, scalable	memory/compute heavy

Common Failure Modes

- **Leakage:** preprocessing fitted on all data, peeking at test labels, target leakage from future information.
- **Metric mismatch:** optimizing accuracy on imbalanced data instead of recall/F_1/PR-AUC.
- **Unscaled features:** breaks distance and gradient-based methods.
- **Wrong validation protocol:** random splits on time series/grouped data when temporal/group leakage exists.
- **Overtuning CV:** too many repeated tweaks on same validation set causes validation overfit.

Quick Recipes

- **Small tabular dataset:** start with logistic/linear regression, trees, random forest, gradient boosting.
- **Distance meaningful + little training time:** try k-NN after scaling.
- **High-dimensional sparse text:** linear models and linear SVMs are very strong baselines.
- **Images with modest data:** transfer learning with CNNs usually beats handcrafted pipelines.
- **Need interpretability:** linear models and shallow trees first.
- **Need calibrated probabilities:** logistic regression or post-hoc calibration.

Rapid Topic Review

Introduction to ML

- ML learns from experience E on tasks T under performance measure P.
- Core inversion from classical programming: data + targets produce the program/model.
- Always separate **representation choice** from **model choice**.

ML Tasks

- Classification predicts a discrete region/label; regression predicts a continuous response.
- Unsupervised learning uses similarity structure instead of labels.
- RL handles sequential decision-making with delayed rewards and exploration.

Supervised Workflow

- Inspect class balance, missingness, scale, outliers, and leakage sources before modeling.
- Hypothesis classes define which patterns are even representable.
- Consistency on train is not enough; validation determines whether the design worked.

Generalization Error

- Empirical risk is computable; true risk is an expectation under the unknown data distribution.
- Validation/test loss are statistical estimates of generalization error.
- Regularization, model-capacity control, and better data help shrink the train–test gap.

Regression and Cross-Validation

- End-to-end regression recipe: choose features, choose loss, fit parameters, validate complexity.
- Overfitting often appears after a sweet spot in model complexity or training duration.
- K-fold CV reduces split variance compared to a single hold-out split.

Ordinary Least Squares

- Design matrix rows are examples, columns are features plus an optional bias column.
- The normal equations solve least squares exactly when $X^\top X$ is invertible.
- OLS is sensitive to outliers and multicollinearity.

k-NN

- Instance-based/lazy learner: defer computation until query time.
- Local method: prediction quality depends on the neighborhood geometry.
- Works best when a meaningful distance metric exists and data dimension is moderate.

Outliers

- Outliers can be global, contextual, or collective.
- Visual tools: histograms, box plots, scatter plots.
- Robust methods prefer medians/MAD/IQR over means/standard deviations.

Scaling

- Standardization centers at 0 with unit variance; robust scaling centers at median with IQR scale.
- Fit scalers on train only, then transform validation/test with the same parameters.
- Scaling changes optimization geometry and distance meaning, not the raw information content.

Correlation and Polynomial Features

- One-hot is the standard encoding for nominal categories.
- Polynomial features convert nonlinear input relationships into a linear-in-weights problem.
- Correlation matrices are diagnostic tools, not causal explanations.

Image Data I: Representation and Filters

- Images can be summarized globally (means, histograms) or locally (patches, filters).
- Filtering computes weighted local sums; kernels detect blur, sharpening, and edges.
- Gradient features capture boundaries and texture changes.

Image Data II: Thresholds and Local Features

- Thresholding turns grayscale into binary masks for segmentation.
- Harris finds corners where intensity changes strongly in two directions.
- SIFT and ORB are classic local descriptors for matching and geometric correspondence.

Image Data III: HOG and Classical Pipelines

- HOG summarizes local edge directions with normalization for illumination robustness.
- Template matching is simple but brittle under scale/rotation changes.
- Classical vision pipeline: preprocess, extract features, normalize, classify with a standard ML model.

Linear Models

- Linear models produce a score $\mathbf{w}^\top \phi(\mathbf{x}) + b$ on features or transformed features.
- Regression/classification differ mostly in output interpretation and loss.
- ERM + regularization is the common training principle.

Gradient Methods

- Gradients point uphill; gradient descent steps downhill.
- Batch size trades variance against compute efficiency.
- Adaptive optimizers are practical defaults, but learning-rate choice still matters.

SVMs

- SVMs maximize margin, not probability fit.
- Soft-margin parameter C balances margin width against training violations.
- Kernels implicitly lift data into richer feature spaces.

Logistic Regression

- Logistic regression is a linear score model with probabilistic interpretation.
- Cross-entropy is the natural loss from Bernoulli/multinomial likelihoods.
- Decision thresholds can be tuned independently of model training.

Decision Trees and Ensembles

- Trees greedily choose splits maximizing impurity/variance reduction.
- Deep trees memorize; pruning and ensembling reduce variance.
- Random forests usually outperform a single tree while sacrificing some interpretability.

MLP Basics

- Hidden layers act as learned feature extractors.
- Nonlinear activations are what make depth useful.
- Width/depth choices control expressivity and optimization behavior.

MLP Training and Softmax

- Backpropagation efficiently computes all gradients via repeated chain rule application.
- Softmax + cross-entropy is the standard multi-class pairing.
- Training curves should be read jointly: train loss, validation loss, and validation metric.

CNNs

- Local connectivity and weight sharing drastically cut parameter count.
- Pooling/downsampling trades spatial precision for abstraction and efficiency.
- Transfer learning is often the strongest baseline on moderate image datasets.

Transformers and GenAI

- Self-attention lets each token condition on the whole context.
- BERT-style encoders are for understanding; GPT-style decoders are for generation.
- Prompting changes conditional generation without changing model weights.

Practical LLM/GAN Notes

- Tokenization quality strongly affects downstream NLP behavior.
- Fine-tuning adapts pretrained representations instead of learning from scratch.
- GAN training is unstable because generator and discriminator co-evolve.

Exam-Style Facts to Remember

1. Simpler models often win with limited data; flexibility needs regularization and validation.
2. Correlation measures linear association only; nonlinear dependence can hide behind $r \approx 0$.
3. Tree-based models are usually invariant to monotone rescaling; distance/margin/gradient models are not.
4. One-hot encoding is safe for nominal categories; label encoding is not unless order is real.
5. Ridge shrinks all coefficients; lasso can perform feature selection.
6. Logistic regression is linear in features but nonlinear in

probability via the sigmoid.

7. SVM support vectors are the informative training points; non-support vectors do not affect the margin solution much.
8. Deep nets learn features automatically, but good optimization and regularization are essential.
9. CNNs beat dense nets on images because locality + weight sharing encode the right inductive bias.
10. Transformers succeed because attention models long-range dependence in parallel.

Formula Bank

$$L_{\text{emp}} = \frac{1}{N}\sum_{i=1}^{N} \ell\big(f(\mathbf{x}_i), y_i\big)$$

$$\text{SSE} = \sum_i (y_i - \hat{y}_i)^2, \qquad \text{MSE} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{MAE} = \frac{1}{N}\sum_i |y_i - \hat{y}_i|, \qquad z = \frac{x-\mu}{\sigma}, \qquad x'_{\text{std}} = \frac{x-\mu}{\sigma}$$

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \log\frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b$$

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}, \qquad \ell_{\text{CE}} = -\sum_k y_k \log p_k$$

$$\ell_{\text{hinge}} = \max(0, 1 - yf(\mathbf{x})), \qquad \gamma_i = \frac{y_i(\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

$$\hat{\mathbf{w}}_{\text{OLS}} = (X^\top X)^{-1} X^\top \mathbf{y}, \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t)$$

$$\hat{y} = w_0 + \sum_j w_j x_j, \qquad r_{xy} = \frac{\text{cov}(x,y)}{\sigma_x \sigma_y}$$

$$H(S) = -\sum_c p_c \log_2 p_c, \qquad G(S) = 1 - \sum_c p_c^2$$

$$IG(S, \text{split}) = H(S) - \sum_j \frac{|S_j|}{|S|} H(S_j)$$

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \quad \text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}$$

$$F_1 = \frac{2PR}{P+R}, \qquad \text{IQR} = Q_3 - Q_1, \qquad x'_{\text{robust}} = \frac{x - \text{median}}{Q_3 - Q_1}$$

$$L_{\text{ridge}} = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_2^2, \qquad L_{\text{lasso}} = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_1$$

$$\delta^{(\ell)} = \big(W^{(\ell+1)\top}\delta^{(\ell+1)}\big) \odot \phi'(a^{(\ell)})$$

$$\frac{\partial L}{\partial W^{(\ell)}} = \delta^{(\ell)} h^{(\ell-1)\top}$$

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

$$\text{conv output size} = \left\lfloor \frac{n+2p-f}{s} \right\rfloor + 1$$